

By Richard Kluin,

Aidan Fuhrmann,

Beiming Gu

Group Project Report

Motivation:

For our group project we chose to create a blogging website where users are able to make text posts that are viewable to other users. We chose this idea for a number of reasons. First of all, it was simple to imagine the three modules we would need to implement: users, posts, and interactions/comments. Another reason is that we all were familiar with the concept of a blogging website and thus we could easily intuit the necessary elements to implement.

Modules:

As previously stated, our project consists of a user, posts, and interactions modules; all three are stored using mongoDB collections. For the users module consists of user objects which only store a username and hashed password. Whenever someone logs in, the password they entered is hashed and the database is searched for an object with matching username and hashed password values. If such a user object is found, then that person is logged in, otherwise the log in is invalid. When a new user is registered, their information is simply stored in a new user object and appended to the users collection. Session management is done through tokens. Users also can customize their blogs in the account page, which changes the site as they navigate, as well as for other users visiting their blog, where they can customize the background color, font color, and font size.

The posts module stores all of the posts made by the users. Each post object in the collection stores the post's title, content, whether it was published or is a draft, the time it was

created, the time it was last updated, and lastly the username of the user who posted it. When a user is logged in they are able to create a post. A user can mark a post as a draft, meaning when they save it, it will not be able to be seen by other users on the home page. A post can be edited after it has already been posted, where a user can change the title and/or content of the post. This updates the last updated time of the post stored in the database.

The interactions/comments module stores all of the comments made by users. Each comment object in the collection stores the comment's content, the username of the person who made it, the time it was created, and the id of the post which it was commented under. When a post is viewed, the interactions collection is searched for all comments that store a reference to that post's id, and are then displayed under the post body.

Functionalities:

Our blogging website allows for users to view posts made by other people and create their own. Appended to every page is a header that contains a link to the multiple pages of the website along with a link to the user's account page if they are logged in. This displays whether a user is logged in and to what account.

Home Dashboard Account	Login Home Dashboard Account	rgekluin2 Logout
--	--	--

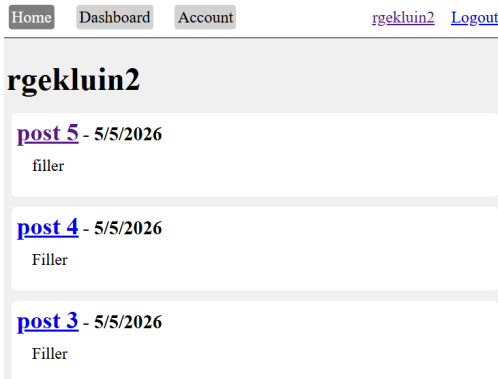
The home page displays all of the posts in the database sorted in chronological order of when they were created. Clicking on the title of a post brings a user to the post's dedicated page where the comments are shown under the post's content. If the currently logged in user is the same as the user who created the post, then a link to a page where the user can edit that post appears.



The dashboard page displays all the posts made by the user, including drafts, along with all their comments. For each post, the title, link to the post's page, create and last updated date and time are shown, along with links to the edit page, and an option to delete the post are shown. This is where a user is able to manage all of their posts. To the right of the posts section is the comments section which shows the posts the user has commented on, the date the comment was created, and a link to view the post and the comments under it. Under both of these sections is the most important button, the “new post” button, which takes the user to a post creation page.



The account page simply displays the number of posts a user has made. Clicking on the username section of any post, as well as a user's own username in the header, goes to an individual user's page which displays all of that user's posts.



To handle hydrating the page with content, as we haven't gone over cookie-based auth and thus couldn't check login status per-request, instead we serve a skeleton that then handles its own hydration via a series of fetch calls to the necessary endpoints. This was done because we can pull from local storage after a page is loaded to send the token and username with the request when needed. The token storage is an in memory map from username to generated session token, and we created a helper function *isloggedin* to be used in any sensitive route that checks the request's username and token headers against the map, and if invalid returns false for the caller to send a response accordingly.

Technical Details:

- Persistent storage of information using mongoDB
- Session management done through tokens
 - Randomly assigned token stored in window's local storage
 - Checked against tokens mapped to usernames in memory
- Backend coded using express, using RESTful api